

CYGO (Cylinder Gonnnect) — Verification Harness & Measured-Strength Evidence

Task games/cygo_superhuman_agent_001. Reference = cygo-run2 global_step_977400 (more-trained of two independent AlphaZero seeds; the other, cygo-run1, reached step 712200). All logs below are from actual runs on the training GPU clusters during development. Generated 2026-09-16.

1. Grading protocol (what the referee does)

An independent referee (scripts/match_harness.py, shipped) plays 1000 games (500 as each color) between the submitted output/run.sh and the HIDDEN reference agent, from forced balanced 6-ply openings, under the exact stdin/stdout protocol and per-move wall-clock budget (3.0 s) on one 24 GB GPU. Rules are authoritative via the shipped C++ engine: cylinder column-wrap, Go capture, suicide illegal, positional superko, win by top-edge to bottom-edge connection, no-legal-move loss, ply-cap-600 stone-count adjudication (ties -> White). Any illegal move / malformed reply / crash / clock or memory-cap violation = loss of that game. Score = clip(2 * candidate_win_rate, 0, 1): equal strength -> 1.0, ~ -520 Elo -> 0.10. Everything is mechanical; self-reported results are ignored; the reference is staged only at eval time.

2. Harness end-to-end run (match_harness.py)

A real match through the shipped referee: a random-move candidate vs the trained reference (MCTS). Confirms the harness runs end-to-end, the reference plays only legal moves to terminal, and scoring is correct (a random candidate is crushed → score 0).

```
$ python3 match_harness.py \
  --candidate "python3 agent_proto.py --mode random" \
  --reference "python3 agent_proto.py --mode mcts --weights weights.pt --ch 256 --
blocks 15 --sims 64" \
  --games 6 --budget 5.0 --out match.json

{ "cand_winrate": 0.0, "score": 0.0, "budget_s": 5.0, "wall_s": 117.1,
  "records": [
    {"game": 0, "cand_color": 1, "winner": 2, "win": 0, "reason": "terminal"},
    {"game": 1, "cand_color": 2, "winner": 1, "win": 0, "reason": "terminal"},
    {"game": 2, "cand_color": 1, "winner": 2, "win": 0, "reason": "terminal"},
    {"game": 3, "cand_color": 2, "winner": 1, "win": 0, "reason": "terminal"},
    {"game": 4, "cand_color": 1, "winner": 2, "win": 0, "reason": "terminal"},
    {"game": 5, "cand_color": 2, "winner": 1, "win": 0, "reason": "terminal"} ] }
```

Reference won all 6 (both colors), every game clean-terminal, no crashes/timeouts. This smoke used an earlier checkpoint (step 165200); the shipped reference (step 977400) is strictly stronger.

3. Measured strength (batched_match.py; 120 games, 128 sims, noise 0.0)

Elo from win-rate via $-400 \cdot \log_{10}(1/\text{wr} - 1)$. Same network architecture on both sides (ch=256, blocks=15). Run on a held GPU.

Match	A win-rate	Elo	What it shows
512 sims vs 64 sims (same net)	0.890	+363	Deep search decisively beats shallow → MCTS tree/backup is correct
c2 step 735350 vs early step 272900	0.983	+708	Genuine large improvement over the run (not churning)
c1 step 218k vs frozen 165k (sims-256 window)	0.592	+64	Higher self-play sims broke the earlier plateau
MCTS / policy vs uniform-random	~1.00	—	Reference dominates a random baseline

```
MATCH SIMS_SCALING_512v64: A_winrate=0.890 (100g simsA=512 simsB=64) elo=+363 vs self
MATCH c2_cur_vs_pre272900: A_winrate=0.983 (120g sims128) elo=+708 vs
/tmp/c2_old_272900.pt
MATCH c1_218k_vs_165k:      A_winrate=0.592 (120g sims128) elo=+64 vs /tmp/c1_165250.pt
```

4. Durability / reproducibility

Two independent seeds (distinct region, account, RNG) were trained with per-step S3 checkpointing (tracker written last; keep-last-3 by recency). Both auto-resumed across multiple 24-hour instance replacements with no manual state loss. Final durable checkpoints: cygo-run1 step 712200, cygo-run2 step 977400 (shipped).

Honesty note. The results above are real development-time runs, not the full 1000-game grade (which runs mechanically at eval time on the shipped harness). No frontier model has yet been graded against the reference, and a from-scratch reachability run under the agent budget is pending; the <1% difficulty estimate rests on design-time adversarial curation (Fable 5.1 + GPT-6-Astra, 4 rounds, self-estimate 0.06–0.12), stated as an estimate in reference/reference_metrics.json.